웹서비스설계 보고서 작성 양식

JWT 게시판 구현 보고서

01 서비스 소개와 구현 범위

학번: 202313746 이름: 인숙영 제출일: 2026.10.17

서비스 이름 : IT 시스템 장애·트러블슈팅 기록 게시판

기술 조합과 버전 React 19.3.0 / Vite 8.3.4 / Spring Boot 3.5.16 / SQLite JDBC 3.51.3.0 / JJWT 0.13.0; Windows 25H2 / Node.js 24.11.1 / npm 11.6.2 / Java 17.0.20.1

GitHub 저장소 URL : https://github.com/abc202313746/JWT-Assignment.git

소스 코드 기준 : 6ec886435f5909b0d8073be0e2b405cfef25b615

이 서비스는 개발자와 시스템 운영자가 서버 오류/장애의 발생 환경, 증상, 원인 분석, 해결 방법을 기록하고 공유하는 게시판이다. 비슷한 장애를 겪었을 때 기존 글의 제목·본문·작성자를 검색해 사례를 찾는다. 본문에는 발생 환경·증상 및 에러 로그·원인 분석·해결 방법을 기록한다. 로그인 없이 목록과 존재하는 글을 조회할 수 있으며 정상 응답은 200 이다.

로그인 상태/에러 배너/목록과 상세 화면 갱신을 컴포넌트 상태로 나누기 위해 React 를 선택했다. JdbcTemplate 의 SQL 실행과 JJWT 의 토큰 발급/검증 흐름을 코드에서 제어하기 위해 Spring Boot 를 선택했다. 제공 템플릿의 게시판 CRUD·SQLite 저장/PBKDF2 검증은 유지했다. AuthService.java 와 frontend-react/src/api.js 의 인증 미완성 부분에 AI 작성 코드를 반영했다. main.jsx 와 style.css 의 화면을 변경하고 BoardController.java 의 fields()에서 비밀번호 원문 길이를 검사하도록 수정했다. 입력 제한을 위반한 응답은 400 BAD_INPUT 이다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	BoardController.java 의 list()·get(), PostStore.java 의 list()·get()	공식 검사의 무토큰 목록/상세 조회 200; 비로그인 화면의 기존 글 표시
로그인과 JWT 발급	완료	BoardController.java 의 login(), Users.java 의 verify(), AuthService.java 의 issueToken()	정상 로그인 200, Alice 정보·expiresIn 90·가린 토큰 캡처; 잘못된 비밀번호/없는 이메일 각각 401 INVALID_CREDENTIALS
인증과 내 정보 조회	완료	AuthService.java 의 currentUserId()·verifiedUserId(), BoardController.java 의 me()	Alice ID 1·Bob ID 2; 무토큰 401 TOKEN_REQUIRED, 변조 401 INVALID_TOKEN, 만료 401 TOKEN_EXPIRED
본인 글 작성·수정·삭제	완료	BoardController.java 의 create()·update()·delete(), AuthService.java 의 assertOwner()	본인 등록 201/수정 200/삭제 204; Bob 의 Alice 글 수정/삭제 각각 403 FORBIDDEN, 거부 후 재조회 일치

기능	완료 여부	관련 파일	비고 또는 제한
만료·로그아웃·오류 표시	완료	frontend-react/src/api.js 의 request()·clearSession(), main.jsx 의 run()	공백 제목 400 BAD_INPUT; 만료 뒤 로그인 해제; 로그아웃 후 Bearer 헤더 없는 내 정보 요청 401 TOKEN_REQUIRED
DB 데이터 유지	완료	DatabaseConfig.java 의 dataSource(), PostStore.java, schema.sql	서버 재실행 후 5 번 글 제목·본문·작성자 유지, 재조회 200

구조와 변경 내역

브라우저에서 4173 번 preview 화면을 연다. frontend-react/src/api.js 의 request()가 /api 로 요청한다. frontend-react/vite.config.js 의 preview.proxy 가 요청을 8000 번 Spring Boot 로 전달한다. Spring MVC 가 BoardController.java 의 매핑된 메서드를 호출한다. 게시글은 PostStore.java 가 JdbcTemplate 으로 SQLite 의 posts 테이블을 조회하며 정상 조회 응답은 200 이다.

login()은 POST /api/auth/login 으로 email·password 를 전송한다. BoardController.java 의 fields()가 입력을 검사하고 Users.java 의 verify()가 계정을 확인한다. AuthService.java 의 issueToken()이 토큰을 발급한다. 응답을 받은 api.js 의 모듈 변수는 session = { accessToken: data.accessToken, user: data.user }로 바뀐다.

authorizationHeaders()는 session.accessToken 이 있으면 Authorization: Bearer <JWT>를 만든다. request()가 fetch()의 headers 에 이 값을 넣는다. /api/me 와 글 등록·수정·삭제에 사용하며, 로그인 상태의 공개 조회에도 같은 헤더가 붙는다. 보호 API 에 Bearer 헤더가 없거나 형식이 맞지 않으면 401 TOKEN_REQUIRED 이다.

main.jsx 의 save()가 api.update()를 호출하면 PUT /api/posts/{id}로 title·content 를 보낸다. 정상 ID 와 정상 JSON 기준으로 BoardController.java 의 update()는 인증 → 글 조회 → assertOwner() → fields() → DB 변경 순서로 실행한다. PostStore.java 의 update()는 title·content·updated_at 만 바꾼다. 정상 수정은 200, 작성자 불일치는 403 FORBIDDEN 이다.

토큰은 api.js 의 모듈 변수 session 에 저장한다. localStorage 와 sessionStorage 에는 기록하지 않는다. 로그아웃·401 응답·새로고침 이후에는 session = { accessToken: "", user: null }이 된다.

Java 파일 기준 경로: backend-spring/src/main/java/edu/classroom/board/

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
AuthService.java 생성자·issueToken()·currentUserId()·verifiedUserId()·assertOwner()	발급 501, 고정 ID 1, 권한 검사 미완성	Codex 작성 코드 반영. 키·HS256·서명·클레임·계정·작성자 검사 추가. 로그인·만료와 수업 공식 검사 도구 실행은 직접 수행	인증 오류 401, 타인 변경 403 FORBIDDEN
BoardController.java 의 fields()	앞뒤 공백을 제거한 값으로 입력을 검사한다. 비밀번호 반환값은 원문을 사용한다.	Codex 가 수정 코드를 적용. 비밀번호의 빈 값·길이 검사를 원문 기준으로 변경. 이메일·제목·본문은 공백 제거 후 검사를 유지.	비밀번호를 원문 기준 1~128 자로 검사. 빈 문자열 또는 길이 초과는 400 BAD_INPUT.
frontend-react/src/api.js 의 login()·authorizationHeaders()·request()·canEdit()	API 공통 호출·204 처리, 인증 연결 미완성	Codex 작성·수정 코드 반영. session 저장·Bearer 첨부·401 해제·작성자 비교·오류 문구 추가. 제공 코드의 204 분기는 유지	401 이면 session 초기화, 403 이면 유지
frontend-react/src/main.jsx 의 run()·save()·이벤트	로그인 폼·게시판·실패 입력 유지	Codex 수정 코드 반영. 비로그인 저장 차단·프로필과 수정 상태 정리·404 목록 갱신·e.status/e.code 표시. 기존 실패 입력 유지	공백 제목 요청 후 입력 유지, 400 BAD_INPUT

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
frontend-react/의 src/main.jsx·src/style.css·index.html·package.json·package-lock.json	기본 화면·제목·의존성	Claude Code 의 화면 초안을 반영하고 직접 화면 확인. 목록·상세 분할, 본문 구역·코드 표시, 검색·복사, 글꼴 추가	title·content 요청 필드는 유지; 표시 형식 변경
frontend-react/src/style.css 의 반응형 규칙	화면 스타일	Codex 수정 코드 반영 후 직접 너비 확인. 상단 줄바꿈은 1023px 이하, 목록·상세 세로 배치는 899px 이하	900px 상단 바 줄바꿈, 390px 세로 배치
루트 README.md, 이 보고서	실행 설명·보고서	Codex 문서 작성 후 코드·직접 남긴 결과와 대조	기본 포트·실행 명령·실제 증거 경로 기록
.gitignore	의존성·빌드 폴더, data/, .env 등을 Git 추적에서 제외	Codex 수정 내용 반영. 환경 파일·DB 파일·IDE 설정 제외 규칙을 추가했다. 환경설정 예시 파일은 예외로 남김.	비밀 키, 실습 DB 와 로컬 설정이 커밋에 포함되는 것을 막기 위해 추가.
docs/07_로컬실행안내.md	제공 템플릿의 기술별 설치·실행 절차 설명	Codex 가 상단 안내 문단을 추가. 현재 실행 명령은 루트 README.md 를 참조하도록 작성.	선택한 기술과 정리 후 폴더 구조에 맞는 실행 안내 위치를 표시
backend-spring/README.md, frontend-react/README.md	각 시작 코드의 설치·실행 방법 설명	Codex 의 제출 폴더 정리 과정에서 삭제. 현재 실행 절차는 루트 README.md 에 정리	설치·환경변수·빌드·실행 명령의 안내 위치를 루트 문서로 수집.

로컬 실행 기록

운영체제와 버전: Windows 11, 버전 25H2(OS 빌드 26200.9457)를 사용했다.

실행 환경 버전: Windows PowerShell 5.1, Node.js 24.11.1, npm 11.6.2, Java 17.0.20.1 을 사용했다.

새 폴더 설치: GitHub 저장소를 JWT-Assignment-final-check 폴더에 clone 한 뒤 백엔드와 프론트엔드를 설치·빌드·실행했다. M14-clone-backend-build.png 에서 백엔드 빌드 성공과 종료 코드 0 을, M16-clone-frontend-build.png 에서 프론트엔드 설치·빌드의 종료 코드 0 을 확인했다.

환경설정: JWT_SECRET 에는 64 자리 16 진수 난수를 설정했다. 비밀 키 원문은 보고서에 기록하지 않았다. JWT_ISSUER 는 webservice-class, JWT_AUDIENCE 는 board-api 로 설정했다. 만료 검사 후 복구한 TOKEN_TTL 값은 90 이다.

프론트엔드 설치·빌드·실행: frontend-react/에서 npm.cmd ci, npm.cmd run build, npm.cmd run preview 를 차례로 실행했다. 빌드 종료 후 확인한 $LASTEXITCODE 값은 0 이다.

백엔드 빌드·실행: backend-spring/에서 .\mvnw.cmd clean package 를 실행했다. 같은 PowerShell 에서 환경변수를 설정한 뒤 java -jar target\jwt-board-1.0.0.jar 로 실행했다. M15-clone-backend-start.png 에서 8000 번 포트의 서버 기동을 확인했으며, 공식 검사의 GET /api/health 응답은 200 이고 status 는 ok 이다.

SQLite 파일 경로: backend-spring/에서 서버를 실행하면 상대 경로 data/board.sqlite 를 사용한다. 프로젝트 루트 기준 경로는 backend-spring/data/board.sqlite 이다.

4173 번 접속 및 API 연결 확인: 브라우저에서 http://127.0.0.1:4173 에 접속했다. frontend-react/vite.config.js 의 preview.proxy 가 /api 요청을 8000 번 백엔드로 전달한다. 로그인 후 /api/me 에서 계정 정보를 확인했으며, 응답 상태는 200 이다.

서버 재시작 후 데이터 유지: 같은 backend-spring/ 폴더에서 서버를 종료한 뒤 TOKEN_TTL=90 으로 재실행했다. 재시작 전후 게시글 수는 모두 2 개이며, 목록 응답 전체를 비교한 결과 UNCHANGED=True 이다. Alice 의 1 번 글과 Bob 의 2 번 글의 제목·본문·작성자·생성 및 수정 시각이 유지되었다. 재시작 후 목록 조회와 1 번 글 상세 조회의 응답은 각각 200 이다.

증거 위치: 실행 및 수동 확인 캡처는 docs/evidence/manual/에 저장했다. 백엔드 빌드는 M14-clone-backend-build.png, 기동은 M15-clone-backend-start.png, 프론트엔드 설치·빌드는 M16-clone-frontend-build.png, preview 의 API 연결은 M17-clone-preview-api.png 에 기록했다. 재시작 로그·데이터 비교·브라우저 조회는 M28-1-restart-log.png, M28-2-restart-data.png, M28-3-restart-preview.png 에 기록했다.

웹서비스설계 03

핵심 코드 설명

3 1 로그인과 JWT 발급

파일 / 함수: backend-spring/src/main/java/edu/classroom/board/BoardController.java 의 login(), AuthService.java 의 issueToken()이다.

코드 발췌

BoardController.java 64~70 행이다.

```
 @PostMapping("/auth/login")
 public Object login(@RequestBody Map<String, Object> body) {
   var input = fields(body, Map.of("email", 254, "password", 128));
   var user = users.verify(input.get("email"), input.get("password"));
   if (user == null) throw new ApiError(401, "INVALID_CREDENTIALS", "이메일 또는 비밀번호를 확인하세요.");
   return Map.of("accessToken", auth.issueToken(user), "expiresIn", ttl, "user", user);
 }
```

코드 발췌: AuthService.java 74~82 행이다.

```
 public String issueToken(Users.PublicUser user) {
   return Jwts.builder()
```

```
        .subject(Long.toString(user.id()))
        .issuer(issuer)
        .audience().add(audience).and()
        .expiration(Date.from(Instant.now().plusSeconds(ttl)))
        .signWith(signingKey, Jwts.SIG.HS256)
        .compact();
    }
```

동작과 응답 코드 설명

POST /api/auth/login 은 JSON 본문에서 이메일과 비밀번호를 받는다. BoardController.java 의 fields()는 추가 필드, 필수 필드 누락, 문자열이 아닌 값을 거부한다. 이메일은 앞뒤 공백을 제거한 뒤 1~254 자인지 검사한다. 비밀번호는 공백을 제거하지 않은 원문으로 1~128 자인지 검사한다. 빈 문자열은 입력 검사에서 거부한다. 공백 한 글자는 입력 검사를 통과한 뒤 Users.java 의 verify()에서 원문 그대로 대조한다. 입력 조건을 벗어나면 400 BAD_INPUT, 계정 정보가 일치하지 않으면 401 INVALID_CREDENTIALS 이다.

계정 확인을 통과하면 AuthService.issueToken()이 HS256 으로 JWT 를 서명한다. sub 에는 계정 ID 를 문자열로 넣고, iss 에는 webservice-class 를 설정하며, aud 에는 board-api 를 포함한다. exp 는 현재 UTC 시각에 ttl 초를 더한 만료 시각이다. TOKEN_TTL 의 기본값은 90 초이며, 만료 검사에서는 3 초를 사용했다. BoardController.login()은 accessToken, expiresIn, user 를 포함한 JSON 을 반환한다. user 에는 id·name·email 이 포함되며 성공 응답은 HTTP 200 이다.

3 2 브라우저 토큰 저장과 전송

파일 / 함수: frontend-react/src/api.js 의 login(), authorizationHeaders(), request(), clearSession()이다.

코드 발췌

frontend-react/src/api.js 38~45 행이다.

```
export async function login(email, password) {
  const data = await request("/auth/login", {
    method: "POST",
    body: JSON.stringify({ email, password }),
  });
  session = { accessToken: data.accessToken, user: data.user };
  return data.user;
}
```

같은 파일의 6~10 행이다.

```
function authorizationHeaders() {
  return session.accessToken
    ? { Authorization: `Bearer ${session.accessToken}` }
    : {};
}
```

같은 파일의 request() 내부 20~22 행이다.

```
  const data = response.status === 204 ? null : await response.json();
  if (!response.ok) {
    if (response.status === 401) clearSession();
```

동작과 응답 코드 설명

login()은 이메일과 비밀번호를 JSON 문자열로 변환해 /api/auth/login 으로 전송한다. 로그인에 성공하면 응답의 토큰과 계정 정보를 frontend-react/src/api.js 의 모듈 변수 session 에 저장한다. localStorage 와 sessionStorage 에는 기록하지 않는다. 로그인 성공 후 변수 상태는 session = { accessToken: data.accessToken, user: data.user }이다.

authorizationHeaders()는 session.accessToken 이 있을 때 Authorization: Bearer <JWT> 헤더를 만든다. 토큰이 없으면 빈 객체를 반환한다. request()는 이 객체를 fetch()의 요청 헤더에 합친다. 내 정보 조회와 게시글 등록·수정·삭제 요청도 request()를 거친다. 토큰 없이 /api/me 를 호출했을 때 서버 응답은 401 TOKEN_REQUIRED 이다.

request()는 응답 상태가 204 이면 JSON 을 읽지 않고 data 를 null 로 설정한다. 나머지 응답은 JSON 본문을 읽는다. 실패 응답에서는 HTTP 상태를 error.status, JSON 오류 코드를 error.code 에 넣어 예외를 던진다. 타인 글 수정·삭제에 대한 403 FORBIDDEN 응답에서는 clearSession()을 호출하지 않으므로 session 은 기존 값으로 유지된다.

로그아웃 시에는 clearSession()이 토큰과 계정 정보를 지운다. 서버가 401 을 반환했을 때도 request()가 같은 함수를 호출한다. 토큰 만료는 보호 API 요청에 대한 401 TOKEN_EXPIRED 응답을 받은 뒤 처리한다. 새로고침하면 모듈을 다시 불러오므로 로그인 정보가 초기화된다. 이때 변수 상태는 session = { accessToken: "", user: null }이다.

웹서비스설계 03 계속

서버 인증과 작성자 권한

3 3 서버 JWT 검증

파일 / 함수: backend-spring/src/main/java/edu/classroom/board/AuthService.java 의 currentUserId(), verifiedUserId()이다.

코드 발췌

AuthService.java 의 currentUserId() 내부 91~100 행이다.

```
  try {
   try {
    return verifiedUserId(parser.parseSignedClaims(bearer.group(1)).getPayload());
   } catch (ExpiredJwtException e) {
    verifiedUserId(e.getClaims());
    throw new ApiError(401, "TOKEN_EXPIRED", "로그인 시간이 만료되었습니다. 다시 로그인하세요.");
   }
  } catch (JwtException | IllegalArgumentException e) {
   throw new ApiError(401, "INVALID_TOKEN", "유효하지 않은 로그인 정보입니다. 다시 로그인하세요.");
  }
```

같은 파일의 verifiedUserId() 내부 104~111 행이다.

```
  if (!(claims.get("exp") instanceof Number)
      || !issuer.equals(claims.getIssuer())
      || claims.getAudience() == null
      || !claims.getAudience().contains(audience)
      || !(claims.get("sub") instanceof String subject)
      || !subject.matches("[1-9][0-9]*")) {
    throw new ApiError(401, "INVALID_TOKEN", "토큰의 필수 정보를 확인할 수 없습니다.");
  }
```

동작과 응답 코드 설명

currentUserId()는 요청의 Authorization 헤더를 읽는다. BEARER 정규식으로 헤더 형식을 검사한 뒤 토큰 문자열을 추출한다. 헤더가 없거나 Bearer 형식에 맞지 않으면 응답은 401 TOKEN_REQUIRED 이다.

AuthService.java 의 생성자는 verifyWith(signingKey)로 서명 검증 키를 설정한다. 허용 서명 알고리즘은 HS256 으로 제한한다. requireIssuer(issuer)와 requireAudience(audience)로 발급자와 대상 값을 지정한다. currentUserId()의 parser.parseSignedClaims()가 이 설정에 따라 토큰을 검증한다. 서명 변조, 허용하지 않은 알고리즘, 발급자 또는 대상 불일치로 검증에 실패하면 응답은 401 INVALID_TOKEN 이다.

verifiedUserId()는 숫자형 exp, 설정값과 일치하는 iss, 설정된 대상을 포함하는 aud 를 검사한다. sub 는 양의 정수를 나타내는 문자열이어야 한다. 조건을 통과하면 Long.parseLong(subject)로 계정 ID 를 변환한다. 이어서 users.find(userId)로 실제 계정이 있는지 확인한다. 필수 클레임 누락, 값의 형식 오류, ID 변환 실패 또는 존재하지 않는 계정의 응답은 401 INVALID_TOKEN 이다.

만료된 토큰에서 ExpiredJwtException 이 발생하면 verifiedUserId(e.getClaims())를 먼저 호출한다. 나머지 필수 클레임과 계정 존재 검사를 통과한 경우에 만료 오류를 반환한다. 이때 응답은 401 TOKEN_EXPIRED 이다.

모든 검사를 통과하면 currentUserId()는 토큰에서 확인한 계정 ID 를 반환한다. 보호 API 는 이 값을 계정 조회와 작성자 비교에 사용한다. 검증 실패 시 Errors.java 의 api()가 JSON 본문에 code 와 message 를 넣는다. 401 응답에는 WWW-Authenticate: Bearer 헤더를 추가하며 HTTP 상태는 401 이다.

3 4 작성자 권한 검사

파일 / 함수: backend-spring/src/main/java/edu/classroom/board/BoardController.java 의 create(), update(), delete()와 AuthService.java 의 assertOwner()이다.

코드 발췌

BoardController.java 의 update() 97~105 행이다.

```
 @PutMapping("/posts/{id}")
 public Object update(
     @PathVariable String id, @RequestBody Map<String, Object> body, HttpServletRequest req) {
```

```
  long user = auth.currentUserId(req), pid = postId(id);
  var p = existing(pid);
  auth.assertOwner(user, p.authorId());
  var input = fields(body, Map.of("title", 100, "content", 5000));
  return store.update(pid, input.get("title"), input.get("content"));
 }
```

같은 파일의 delete() 내부 109~113 행이다.

```
  long user = auth.currentUserId(req), pid = postId(id);
  var p = existing(pid);
  auth.assertOwner(user, p.authorId());
  store.delete(pid);
  return ResponseEntity.noContent().build();
```

AuthService.java 의 assertOwner() 119~123 행이다.

```
 public void assertOwner(long userId, long authorId) {
  if (userId != authorId) {
   throw new ApiError(403, "FORBIDDEN", "본인이 작성한 기록만 수정하거나 삭제할 수 있습니다.");
  }
 }
```

동작과 응답 코드 설명

글 등록 시 create()는 currentUserId()가 반환한 계정 ID 를 store.create()에 전달한다. 작성자 ID 는 이 값으로 결정한다. 요청 본문에서 허용하는 필드는 title 과 content 이다. 본문에 authorId 나 userId 를 추가하면 fields()가 거부하며, 응답은 400 BAD_INPUT 이다.

update()는 토큰 인증을 먼저 수행한다. 인증을 통과하면 postId()로 글 ID 형식을 검사한다. 이어서 existing()으로 글을 조회한다. 양의 정수 형식의 ID 에 해당하는 글이 없으면 응답은 404 NOT_FOUND 이다.

글이 존재하면 assertOwner(user, p.authorId())를 호출한다. 토큰에서 확인한 계정 ID 와 저장된 글의 작성자 ID 를 비교한다. 두 값이 다르면 입력 내용 검사와 DB 수정을 진행하지 않는다. 응답은 403 FORBIDDEN 이다.

작성자 확인을 통과하면 fields()가 제목과 본문을 검사한다. 앞뒤 공백을 제거한 제목은 1~100 자, 본문은 1~5000 자여야 한다. 필드 누락, 문자열이 아닌 값, 공백만 있는 값, 길이 초과 또는 추가 필드는 400 BAD_INPUT 으로 처리한다. 검사를 통과하면 store.update()가 DB 를 수정하며, 응답은 200 이다.

delete()도 인증, 글 ID 형식 검사, 글 조회, 작성자 비교 순서로 처리한다. 작성자가 일치하면 store.delete(pid)로 글을 삭제한다. 삭제 성공 응답은 본문이 없는 204 이다.

401 은 토큰이 없거나 검증에 실패해 요청 계정을 확인할 수 없는 경우이다. 403 은 토큰 인증을 통과했지만 해당 글의 작성자가 아닌 경우이다. frontend-react/src/api.js 의 canEdit()는 수정·삭제 버튼 표시 여부를 결정한다. API 를 직접 호출한 요청에도 서버의 assertOwner()가 실행되므로 타인 글 변경 요청의 응답은 403 FORBIDDEN 이다.

웹서비스설계 04

테스트 결과

실행 환경: Windows 11 버전 25H2(OS 빌드 26200.9457), Windows PowerShell 5.1, Node.js 24.11.1, npm 11.6.2, Java 17.0.20.1 과 Google Chrome 을 사용했다. 백엔드는 backend-spring/에서 .\mvnw.cmd clean package 후 java -jar target\jwt-board-1.0.0.jar 로 실행했다. 프론트엔드는 frontend-react/에서 npm.cmd ci, npm.cmd run build, npm.cmd run preview 순서로 실행했다.

로컬 배포 확인: GitHub 저장소를 별도 JWT-Assignment-final-check 폴더에 clone 하여 설치·빌드·실행했다. M14-clone-backend-build.png 에서 백엔드 BUILD SUCCESS 와 종료 코드 0 을, M16-clone-frontend-build.png 에서 프론트엔드 설치·빌드의 종료 코드 0 을 확인했다. M17-clone-preview-api.png 에서는 4173 번 preview 를 통한 게시글 목록 조회 200 을 확인했다. M28-1-restart-log.png, M28-2-restart-data.png, M28-3-restart-preview.png 에서는 서버 재시작 후 게시글 2 개의 전체 응답 필드가 유지되고, 목록과 상세 조회가 200 인 것을 확인했다.

테스트 일시 및 코드 버전: 기존 증거는 2026 년 10 월 15~16 일에 수행한 기록이며, 비밀번호 빈 값 검사를 수정하기 전의 결과이다. M14 이후의 새 폴더 설치·빌드, 공식 검사, 추가 API·화면 확인과 서버 재시작은 2026 년 10 월 17 일에 수행했다. 테스트 기준 소스 코드 커밋 SHA 는 6ec886435f5909b0d8073be0e2b405cfef25b615 이다. 검증한 clone 의 HEAD 는 cb012af257d7b9c53c076c374e2494d775060e0a 이며, 두 커밋의 backend-spring/, frontend-react/, tools/check_api.py 내용은 동일하다.

토큰 TTL: 일반 검사는 TOKEN_TTL=90 으로 실행하고 M24-check-api-final.log 에 기록했다. 만료 검사는 TOKEN_TTL=3 으로 서버를 재시작한 뒤 실행하고 M25-check-api-expiry.log 에 기록했다. 4 초 대기 후 내 정보·등록·수정·삭제 요청에서 401 TOKEN_EXPIRED 를 확인했다. 만료 검사와 화면 확인을 마친 뒤 TOKEN_TTL=90 으로 복원하여 서버를 재실행했다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	토큰 없이 게시글 목록과 존재하는 글 상세 조회	목록·상세 각각 200	목록 200 / 해당 없음, 상세 200 / 해당 없음. 비로그인 화면에서 기존 글 표시	PASS	M24-check-api-final.log 의 public list·public detail, M28-3-restart-preview.png

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T02	Alice 계정으로 로그인 후 응답과 요청 헤더 확인	200, JWT 발급·로그인 표시	200 / 해당 없음. 응답에 accessToken·Alice ID 1·expiresIn 90 포함. 후속 요청에 Bearer 헤더 첨부	PASS	M01-login-response-masked.png, M01-1-bearer-request-masked.png, M24-check-api-final.log
T03	잘못된 비밀번호와 존재하지 않는 이메일로 각각 로그인	각각 401 INVALID_CREDENTIALS	두 요청 모두 401 / INVALID_CREDENTIALS	PASS	M24-check-api-final.log 의 wrong password·unknown email
T04	토큰 없이 내 정보 조회와 글 등록 요청	각각 401 TOKEN_REQUIRED	GET /api/me, POST /api/posts 모두 401 / TOKEN_REQUIRED. WWW-Authenticate: Bearer 검사 통과	PASS	M24-check-api-final.log 의 no token me·no token create
T05	Alice 의 정상 토큰으로 글 등록 후 내 정보 조회	등록 201, authorId 1. 내 정보 200, Alice 정보	등록 201 / 해당 없음, authorId 1, Location: /api/posts/1. 내 정보 조회 200 / 해당 없음, Alice ID 1.	PASS	M21-0-alice-post-create.png, M24-check-api-final.log 의 create post·Alice me
T06	Alice 의 정상 토큰으로 본인 글 수정·삭제 후 재조회	수정 200, 삭제 204, 삭제 후 조회 404	수정 200 / 해당 없음, 제목 변경 확인. 삭제 204 / 해당 없음, 빈 본문. 삭제 후 404 / NOT_FOUND	PASS	M02-1-owner-update.png, M24-check-api-final.log 의 owner update·delete owned test post·deleted post missing
T07	Bob 의 정상 토큰으로 Alice 글 수정 후 공개 재조회	403 FORBIDDEN, 원문 유지	PUT 403 / FORBIDDEN. 후속 조회 200 / 해당 없음, 글 전체 필드가 요청 전 값과 일치	PASS	M22-1-bob-update-403.png, M24-check-api-final.log 의 Bob denied PUT·직후 failed request leaves data unchanged
T08	Bob 의 정상 토큰으로 Alice 글 삭제 후 공개 재조회	403 FORBIDDEN, 글 유지	DELETE 403 / FORBIDDEN. 후속 조회 200 / 해당 없음, 글 전체 필드가 요청 전 값과 일치	PASS	M22-2-bob-delete-403.png, M24-check-api-final.log 의 Bob denied DELETE·직후 failed request leaves data unchanged
T09	토큰 서명 부분의 첫 글자를 바꾼 뒤 내 정보·등록·수정·삭제 요청	네 요청 모두 401 INVALID_TOKEN	GET·POST·PUT·DELETE 모두 401 / INVALID_TOKEN. WWW-Authenticate: Bearer 검사 통과. 기존 글 재조회 결과 일치	PASS	M24-check-api-final.log 의 tampered 4 건·후속 failed request leaves data unchanged, M24-check-api-final.png

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T10	TTL 3 초로 토큰 발급 후 4 초 대기. 내 정보·등록·수정·삭제 요청과 브라우저 만료 처리 확인	401 TOKEN_EXPIRED, 로그인 해제·만료 안내	네 API 요청 모두 401 / TOKEN_EXPIRED. 기존 글 재조회 결과 일치. 브라우저에서 만료 안내와 로그인 해제 확인	PASS	M25-check-api-expiry.log, M25-check-api-expiry.png, M27-expired-session-cleared.png
T11	로그인 후 로그아웃한 상태에서 내 정보 조회	Authorization 헤더 없음, 401 TOKEN_REQUIRED	GET /api/me 의 401 / TOKEN_REQUIRED. Request headers 에 Authorization 없음. WWW-Authenticate: Bearer 확인	PASS	M13-1-logout-headers.png, M13-1-logout-response.png
T12	Bob 의 글 등록 본문에 authorId: 1 과 userId: 1 을 각각 추가	각각 400 BAD_INPUT	두 추가 필드 요청 모두 400 / BAD_INPUT 으로 등록 거부	PASS	M24-check-api-final.log 의 reject authorId·reject userId
T13	본인 토큰으로 공백 제목·공백 본문·제목 101 자·본문 5001 자 등록 요청. 공백 제목 수정 요청	각각 400 BAD_INPUT	공식 검사의 네 입력 모두 400 / BAD_INPUT. 수동 공백 제목 수정도 400 / BAD_INPUT, 입력 내용과 로그인 상태 유지	PASS	M24-check-api-final.log 의 input validation 4 건, M21-2-alice-blank-title-400.png, M03-input-400.png
T14	없는 양의 정수 ID 조회·수정·삭제와 잘못된 ID 조회. 수정·삭제에는 정상 토큰 사용	없는 글 404 NOT_FOUND, 잘못된 ID 400 BAD_INPUT	ID 9007199254740991 의 GET·PUT·DELETE 모두 404 / NOT_FOUND. ID abc 의 GET 은 400 / BAD_INPUT	PASS	M24-check-api-final.log 의 missing ID·missing protected PUT·missing protected DELETE·bad ID, M20-2-missing-valid-token-404.png
T15	글 저장 후 서버 프로세스를 종료·재실행한 뒤 기존 글 조회	제목·본문·작성자 유지	재시작 후 목록 조회 200 / 해당 없음. 게시글 수는 재시작 전후 모두 2 개이며 전체 응답 필드 비교 결과 UNCHANGED=True. 1 번 글 상세 조회도 200 / 해당 없음.	PASS	M28-1-restart-log.png, M28-2-restart-data.png, M28-3-restart-preview.png
T16	로그인한 상태에서 페이지 새로고침 후 기존 글 조회	로그인 해제, 공개 글 유지	화면에 로그아웃 상태 표시, 작성 입력 비활성화. 기존 5 번 글 조회 200 / 해당 없음	PASS	M05-refresh.png

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T17	토큰 없이 수정·삭제 요청. Bob 의 정상 토큰으로 본인 글 등록·수정·삭제	무토큰 각각 401 TOKEN_REQUIRED. 본인 작업 201·200·204	무토큰 PUT·DELETE 모두 401 / TOKEN_REQUIRED. Bob 등록 201 / 해당 없음, 수정 200 / 해당 없음, 삭제 204 / 해당 없음, 빈 본문	PASS	M24-check-api-final.log 의 no token PUT·DELETE·Bob own update·삭제 검사, M04-1-bob-create.png, M04-2-bob-update.png, M04-3-bob-delete.png

HTTP 응답 확인 기록

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	**200**	공개 목록·상세, 정상 로그인, 내 정보 조회, 본인 글 수정 모두 200 / 해당 없음	M24-check-api-final.log, M28-3-restart-preview.png
새 글 등록	**201 및 Location**	Alice 글 등록 201 / 해당 없음, authorId 1, Location: /api/posts/1	M21-0-alice-post-create.png, M24-check-api-final.log
본인 글 삭제	**204 및 빈 본문**	204 / 해당 없음, 빈 본문. 화면에서 삭제한 글이 목록에서 제거됨. 공식 검사의 삭제 후 재조회는 404 / NOT_FOUND	M04-3-bob-delete.png, M24-check-api-final.log
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	공백 제목·본문, 제목 101 자·본문 5001 자, authorId·userId 추가 요청 모두 400 / BAD_INPUT. 빈 문자열 비밀번호도 400 / BAD_INPUT	M24-check-api-final.log, M21-2-alice-blank-title-400.png, M18-password-inputs.png
잘못된 계정	**401 INVALID_CREDENTIALS**	잘못된 비밀번호와 없는 이메일은 각각 401 / INVALID_CREDENTIALS. 공백 1 자 비밀번호와 정상 비밀번호 앞뒤에 공백을 추가한 입력도 401 / INVALID_CREDENTIALS	M24-check-api-final.log, M18-password-inputs.png

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	무토큰 내 정보·등록·수정·삭제 요청은 모두 401 / TOKEN_REQUIRED. GET /api/me 에 Authorization: Basic abc 를 보낸 요청도 401 / TOKEN_REQUIRED 이며 WWW-Authenticate: Bearer 를 확인.	docs/evidence/manual/M24-check-api-final.log, docs/evidence/manual/M19-auth-header-format.png
변조·클레임 오류	**401 INVALID_TOKEN**	서명 변조 토큰의 내 정보·등록·수정·삭제 요청은 모두 401 / INVALID_TOKEN. 잘못된 알고리즘, iss/aud 불일치, exp/sub 누락 조건을 따로 발급한 시험은 미실행.	docs/evidence/manual/M24-check-api-final.log
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	TTL 3 초 토큰 발급 후 4 초 대기. 내 정보·등록·수정·삭제 요청 모두 401 / TOKEN_EXPIRED. 브라우저에서 만료 안내와 로그인 해제 확인	M25-check-api-expiry.log, M25-check-api-expiry.png, M27-expired-session-cleared.png
다른 작성자의 변경	**403 FORBIDDEN**	Bob 의 정상 토큰으로 Alice 글 수정·삭제 시 각각 403 / FORBIDDEN. 재조회 200 과 UNCHANGED=True 확인. 화면의 403 처리 후 Bob 로그인 유지	M22-1-bob-update-403.png, M22-2-bob-delete-403.png, M23-1-forbidden-session.png
없는 글	**404 NOT_FOUND**	ID 9007199254740991 의 공개 조회와 정상 토큰 수정·삭제 요청 모두 404 / NOT_FOUND	M24-check-api-final.log 의 missing ID·missing protected PUT·missing protected DELETE, M20-2-missing-valid-token-404.png

공식 검사 도구의 check()는 무토큰·서명 변조·만료 요청의 실제 HTTP 상태와 JSON code 를 검사한다. 보호 API 의 401 응답에서는 WWW-Authenticate 값이 Bearer 로 시작하는지도 검사한다. M24-check-api-final.log 에서 일반 검사 40 개, M25-check-api-expiry.log 에서 만료 포함 검사 48 개의 통과와 종료 코드 0 을 확인했다. M19-auth-header-format.png 에서는 GET /api/me 에 Authorization: Basic abc 를 보내 401 TOKEN_REQUIRED 와 WWW-Authenticate: Bearer 를 확인했다.

M27-expired-session-cleared.png 에서는 TTL 3 초로 로그인한 뒤 만료 후 내 정보 조회를 요청했다. 서버가 401 TOKEN_EXPIRED 를 반환했고, 화면은 로그아웃 상태로 바뀌었다. M23-1-forbidden-session.png 에서는 Bob 으로 로그인한 상태에서 브라우저 콘솔을 이용해 한 번의 DELETE 요청 대상만 본인 글 2 번에서 Alice 글 1 번으로 바꾸었다. 실제 서버의 403 FORBIDDEN 응답을 frontend-react/src/api.js 의 request()와 main.jsx 의 run()이 처리한 뒤, 권한 안내 배너가 표시되고 Bob 의 로그인 상태는 유지되었다.

공식 검사 도구의 unchanged()는 거부된 변경 요청 이후 공개 GET 으로 같은 글을 재조회하고, 응답 객체 전체를 요청 전 객체와 비교한다. M24·M25 로그에서 타인 글 변경, 서명 변조, 토큰 만료 요청 이후의 비교가 통과했다. M22-1-bob-update-403.png 와 M22-2-bob-delete-403.png 에서도 Bob 의 수정·삭제 요청이 각각 403 FORBIDDEN 으로 거부된 뒤, 재조회 HTTP 200 과 UNCHANGED=True 를 확인했다. 비교 대상에는 제목·내용·작성자·생성 및 수정 시각이 포함된다.

없는 글 ID 9007199254740991 에 같은 정상 JSON 으로 수정 요청을 보냈다. M20-1-missing-no-token-401.png 의 무토큰 요청은 401 TOKEN_REQUIRED 이고, M20-2-missing-valid-token-404.png 의 정상 Alice 토큰 요청은 404 NOT_FOUND 이다. BoardController.update()는 currentUserId()로 인증한 뒤 existing()으로 글을 조회하므로, 인증 여부에 따라 서로 다른 단계에서 요청이 거부된다.

M21-1-bob-blank-title-403.png 에서는 Bob 의 정상 토큰으로 Alice 글에 공백 제목을 보내 403 FORBIDDEN 을 확인했다. M21-2-alice-blank-title-400.png 에서는 Alice 가 본인 글에 공백 제목을 보내 400 BAD_INPUT 을 확인했다. 두 요청 모두 재조회 결과는 HTTP 200 과 UNCHANGED=True 이다. M26-expired-bob-edit-401.png 에서는 만료된 Bob 토큰으로 Alice 글에 정상 제목과 내용을 보내 401 TOKEN_EXPIRED 를 확인했으며, 재조회 결과도 HTTP 200 과 UNCHANGED=True 이다. 수정 요청의 검사 순서는 인증, 글 조회, assertOwner(), fields(), DB 반영이다.

웹서비스설계 04 계속

테스트 증거

로그인과 토큰 전송

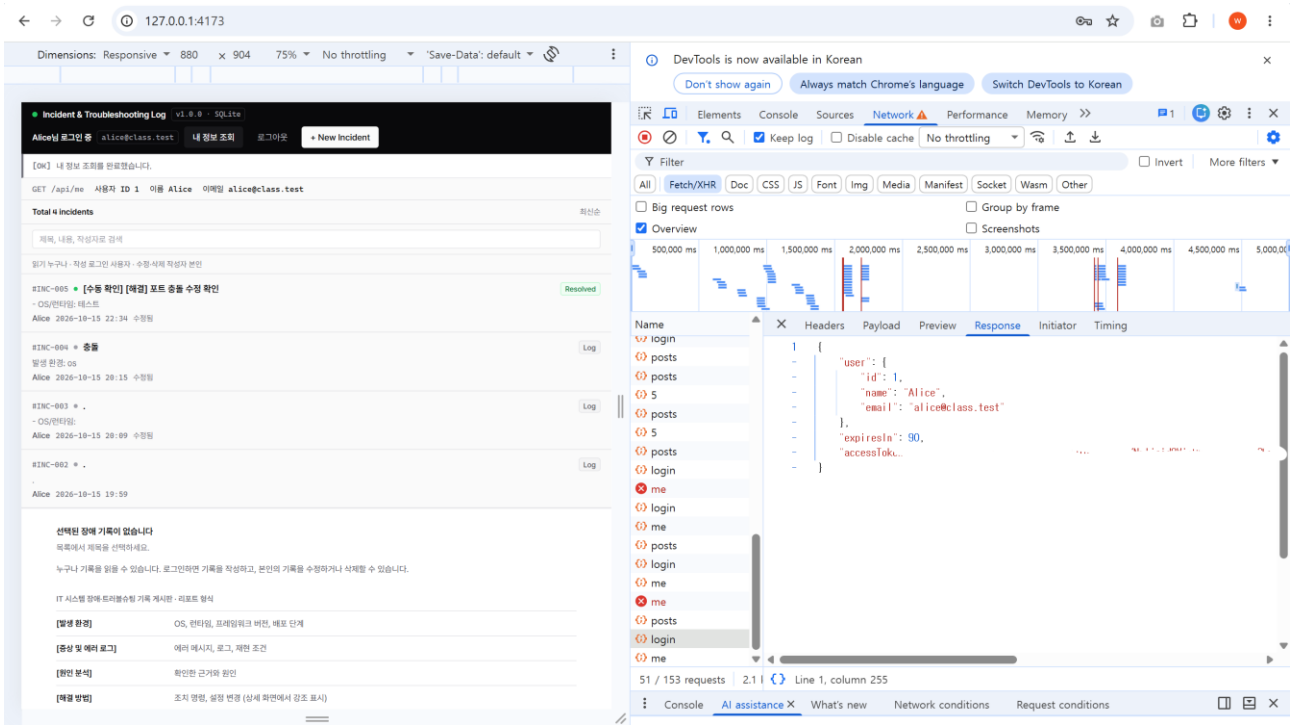


그림 1. T02 — Alice 로그인 성공. 응답 200, expiresIn 90, 토큰 값 마스킹

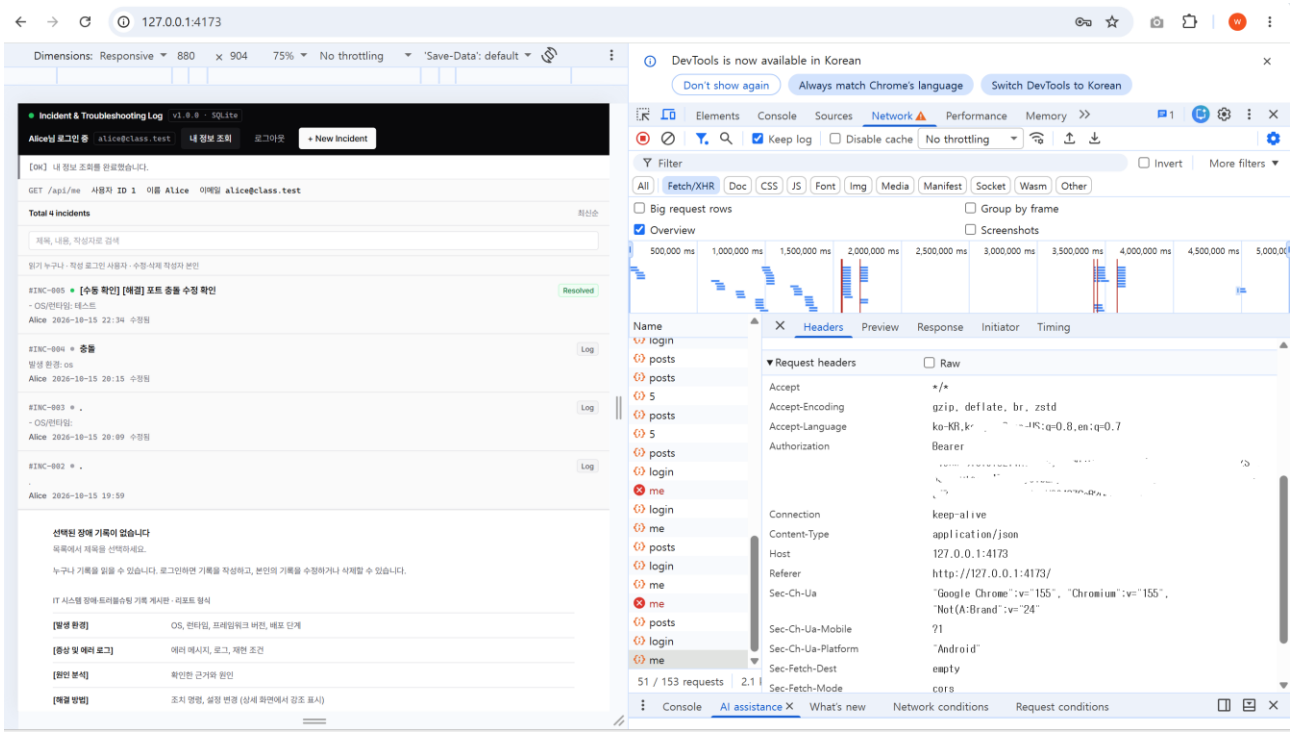


그림 2. T02 — 보호 API 요청의 Authorization: Bearer <JWT> 헤더 확인, 토큰 값 마스킹

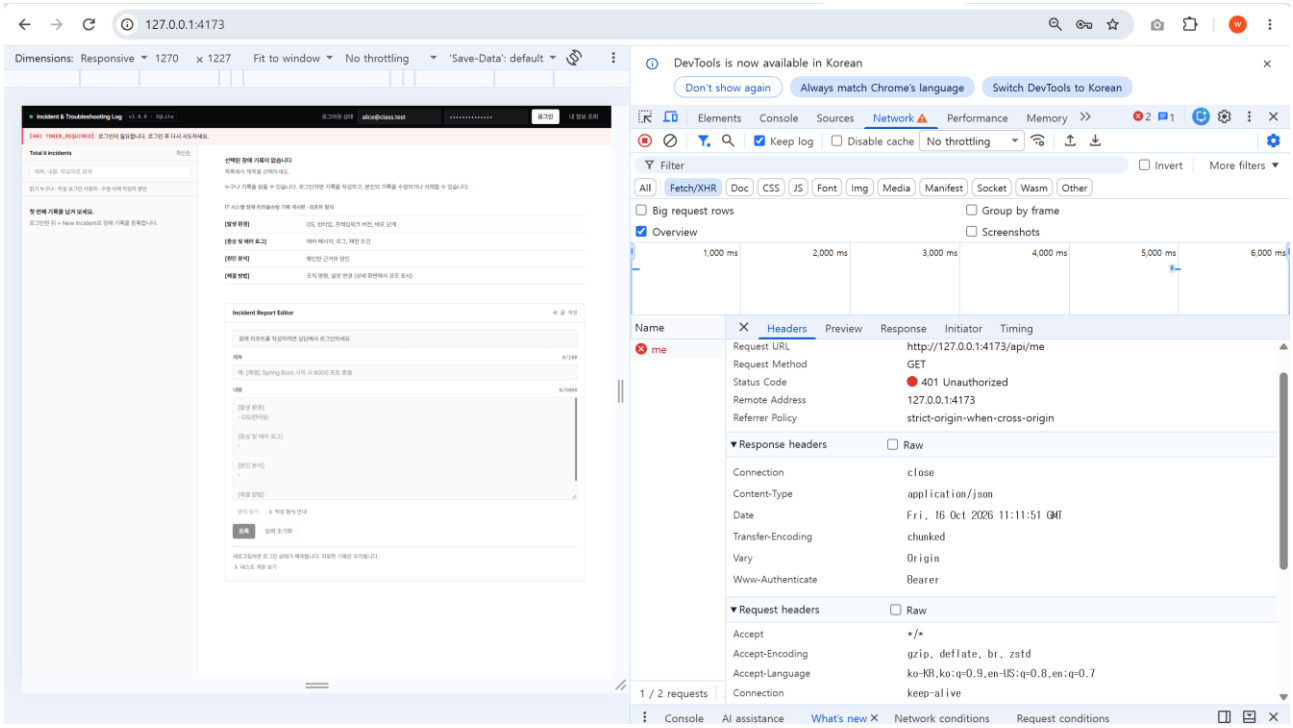


그림 3. T11 — 로그아웃 후 내 정보 요청. Authorization 헤더 없음, 응답 401, WWW-Authenticate: Bearer

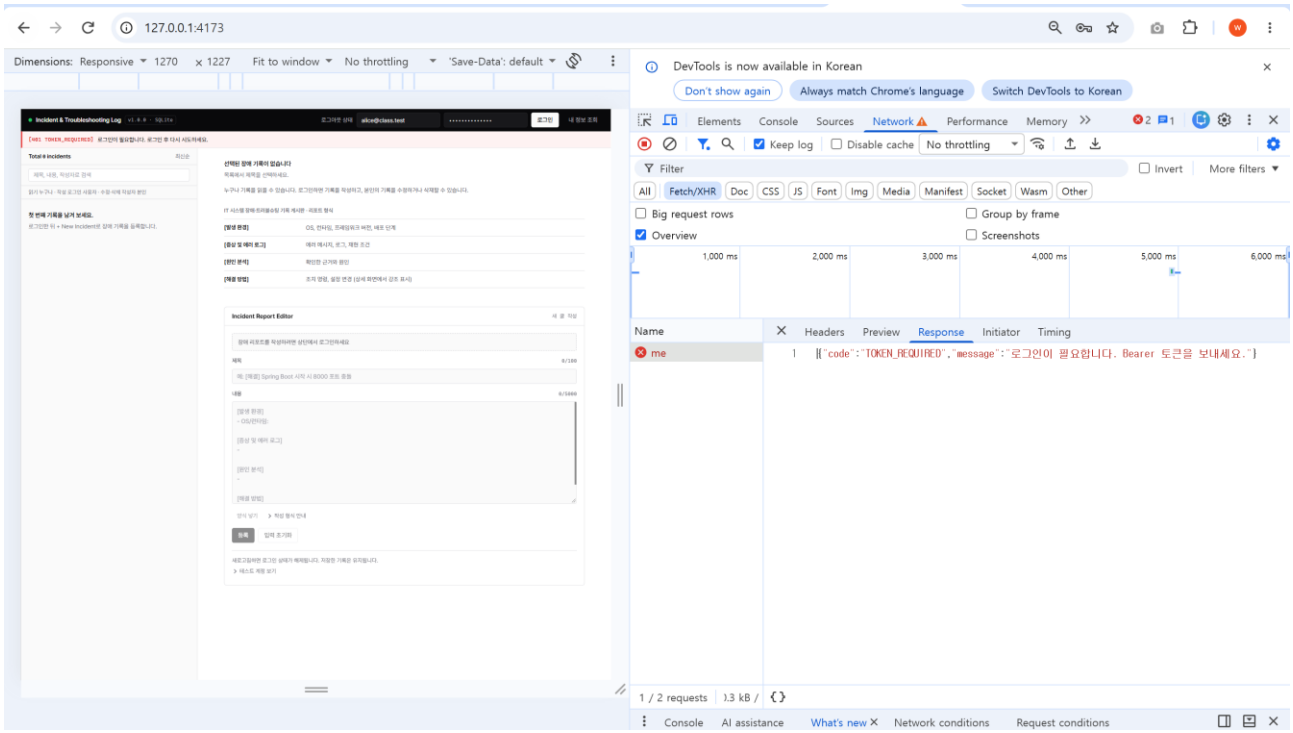


그림 4. T11 — 로그아웃 후 내 정보 응답 JSON 의 TOKEN_REQUIRED 확인

다른 작성자의 수정과 삭제

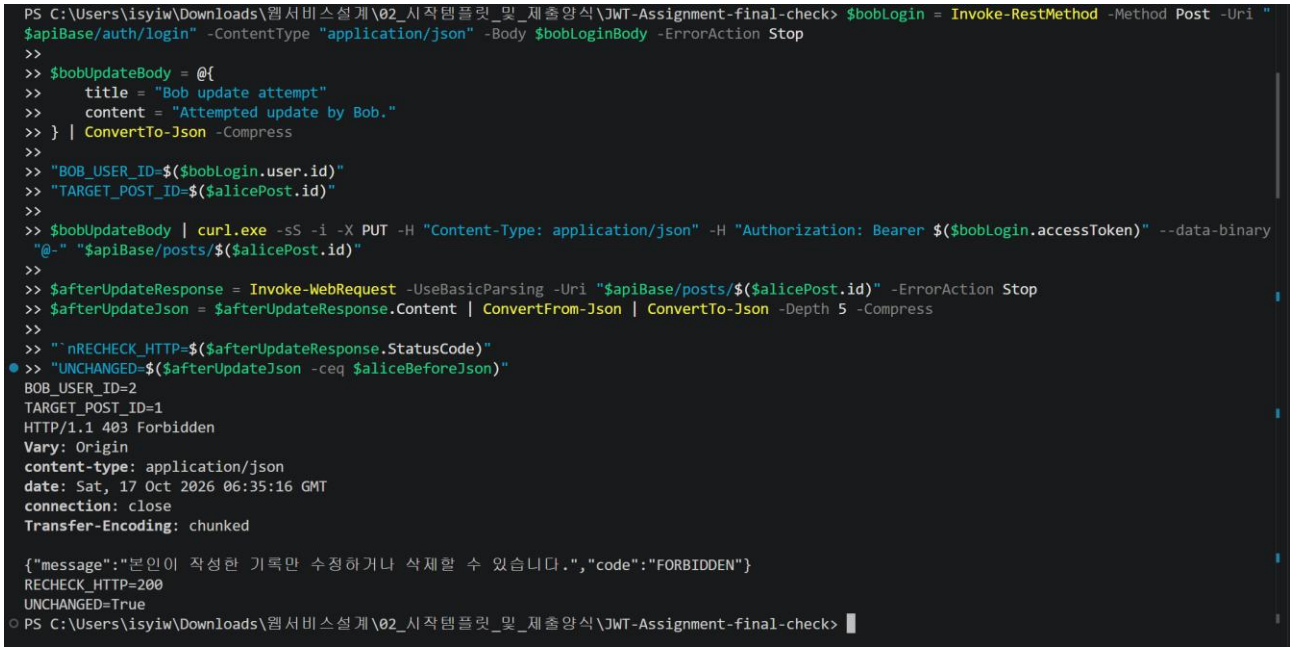


그림 5. T07 — Bob 의 Alice 글 수정 요청. PUT /api/posts/1 은 403 FORBIDDEN, 재조회는 200, UNCHANGED=True

Bob 의 정상 토큰으로 Alice 의 1 번 글에 수정·삭제 요청을 직접 보냈다. 두 응답은 각각 403 FORBIDDEN 이며 공개 재조회에서 HTTP 200 과 UNCHANGED=True 를 확인했다. 화면 처리는 브라우저 콘솔에서 한 번의 DELETE 요청 대상만 Bob 의 2 번 글에서 Alice 의 1 번 글로 바꾸어 확인했다. 실제 서버의 403 응답 후 권한 안내 배너가 표시되고 Bob 의 로그인 상태가 유지되었다.

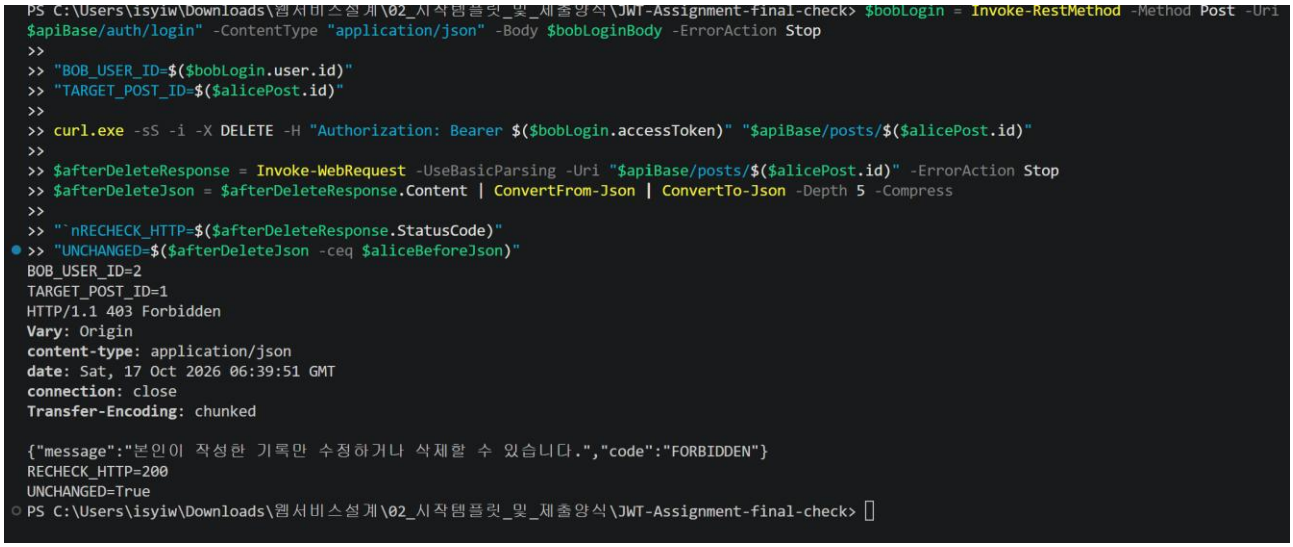


그림 6. T08 — Bob 의 Alice 글 삭제 요청. DELETE /api/posts/1 은 403 FORBIDDEN, 재조회는 200, UNCHANGED=True

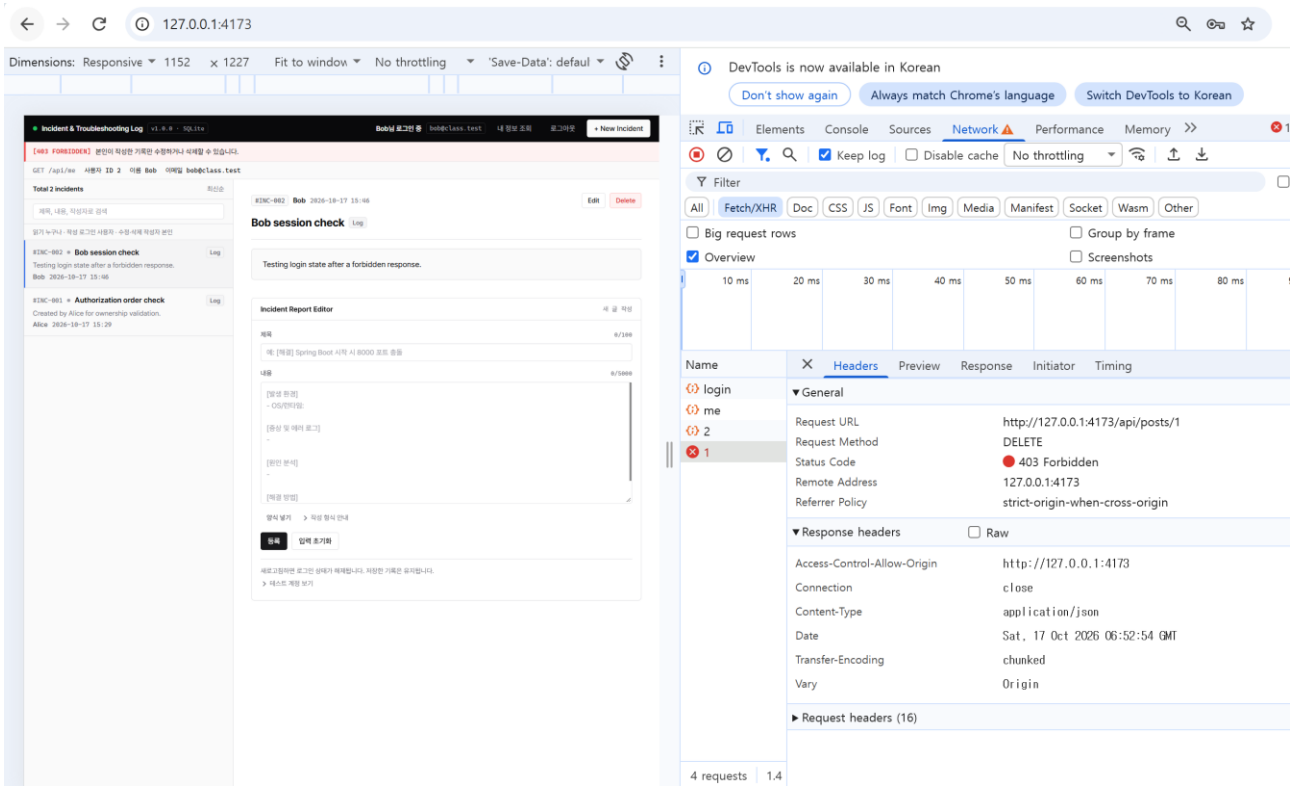


그림 7. 403 응답의 화면 처리 — DELETE /api/posts/1 의 403 응답 후 권한 안내와 Bob 로그인 유지

토큰 변조와 만료

```
>> $expiryLogsyiw\Downloads\웹서비스설계\02_시작템플릿_및_제출양식\JWT-Assignment-final-check>
PASS Bob me
PASS create post
PASS public detail
PASS no token PUT
PASS Bob denied PUT
PASS failed request leaves data unchanged
PASS no token DELETE
PASS Bob denied DELETE
PASS failed request leaves data unchanged
PASS reject authorId
PASS reject userId
PASS input validation
PASS input validation
PASS input validation
PASS input validation
PASS tampered GET /me
PASS tampered POST /posts
PASS tampered PUT /posts/5
PASS tampered DELETE /posts/5
PASS failed request leaves data unchanged
PASS owner update
PASS create post
PASS Bob own update
PASS missing protected PUT
PASS missing protected DELETE
PASS login alice@class.test
Waiting 4 seconds for expiration
PASS expired GET /me
PASS expired POST /posts
PASS expired PUT /posts/5
PASS expired DELETE /posts/5
PASS failed request leaves data unchanged
PASS login alice@class.test
PASS login bob@class.test
PASS delete owned test post
PASS deleted post missing
PASS delete owned test post
PASS deleted post missing
All 48 checks passed (final). UI and restart tests must be performed separately.
ExitCode: 0
```

그림 8. T09·T10 — 서명 변조·만료 포함 공식 검사 48 개 통과. 4 초 대기 후 내 정보·등록·수정·삭제의 만료 검사 통과, 종료 코드 0

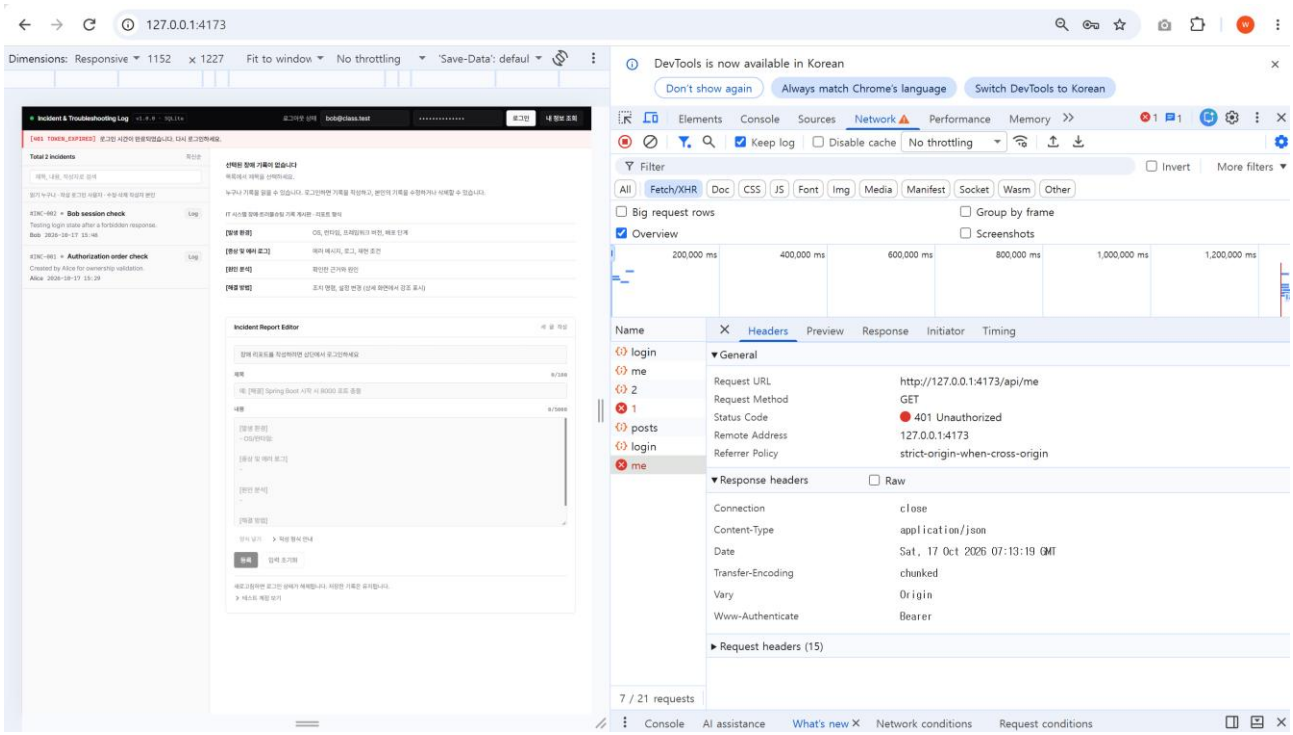


그림 9. T10 — 만료 후 GET /api/me 응답 401 TOKEN_EXPIRED 와 WWW-Authenticate: Bearer 확인. 만료 안내 표시와 로그인 해제

자동 검사와 로컬 배포 증거

수업 공식 검사 도구 tools/check_api.py 를 직접 실행했다. 일반 검사는 TOKEN_TTL=90 으로, 만료 포함 검사는 TOKEN_TTL=3 으로 서버를 재시작한 뒤 수행했다. M25 로그의 Waiting 4 seconds 는 로그인 응답의 expiresIn 3 초에 1 초를 더한 대기다. 만료 확인을 마친 뒤 TOKEN_TTL=90 으로 복원하여 서버를 재실행했다.

py -u tools/check_api.py --mode final

py -u tools/check_api.py --mode final --expiry

M24-check-api-final.log 에서 All 40 checks passed (final).을, M25-check-api-expiry.log 에서 All 48 checks passed (final).을 확인했다. 두 실행의 종료 코드는 모두 0 이며, 전체 출력은 각각의 로그 파일에 UTF-8 로 저장했다. 화면의 403 처리 후 로그인 유지, 401 만료 응답 후 로그인 해제, 서버 재시작 후 데이터 유지는 별도의 수동 확인으로 기록했다.

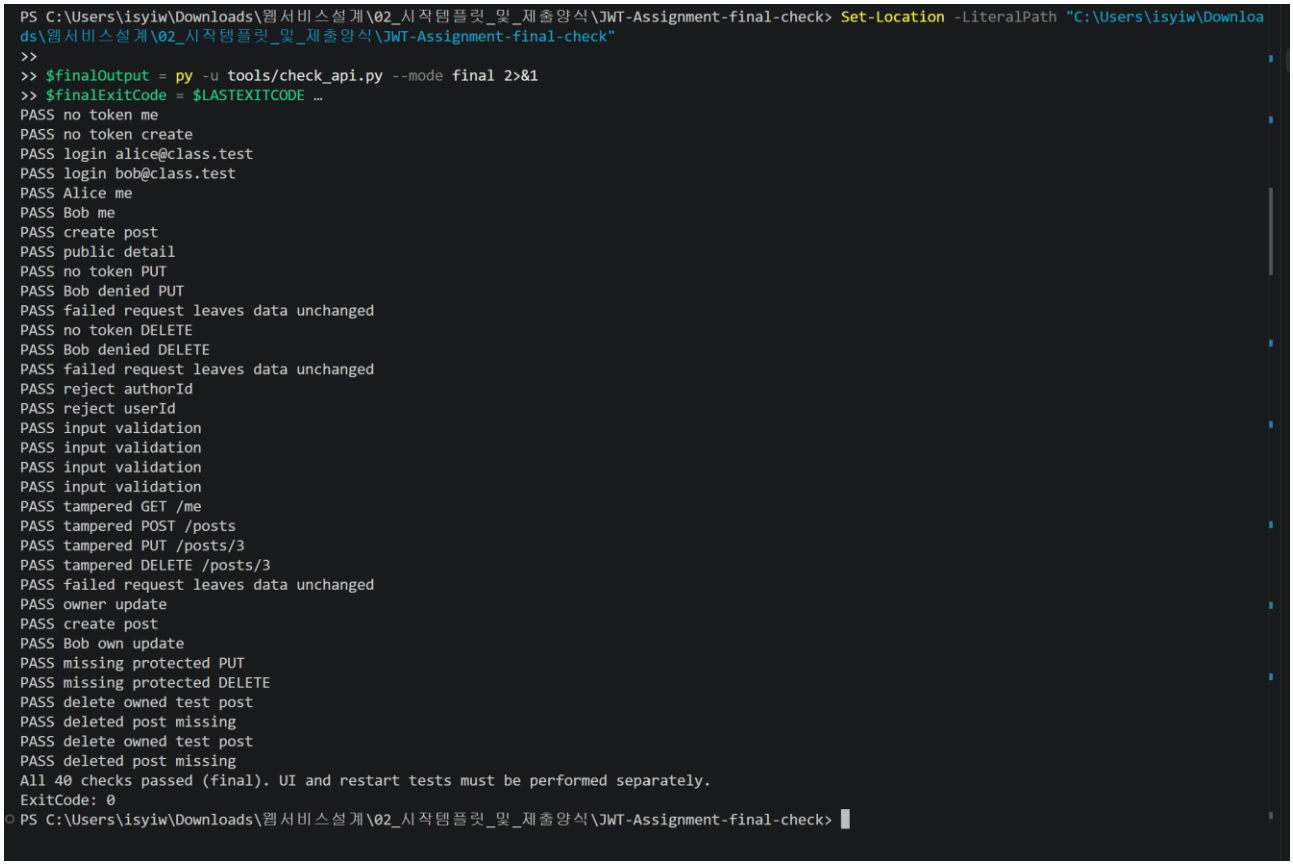


그림 10. 공식 일반 검사 — TOKEN_TTL=90 환경에서 40 개 검사 통과, 종료 코드 0

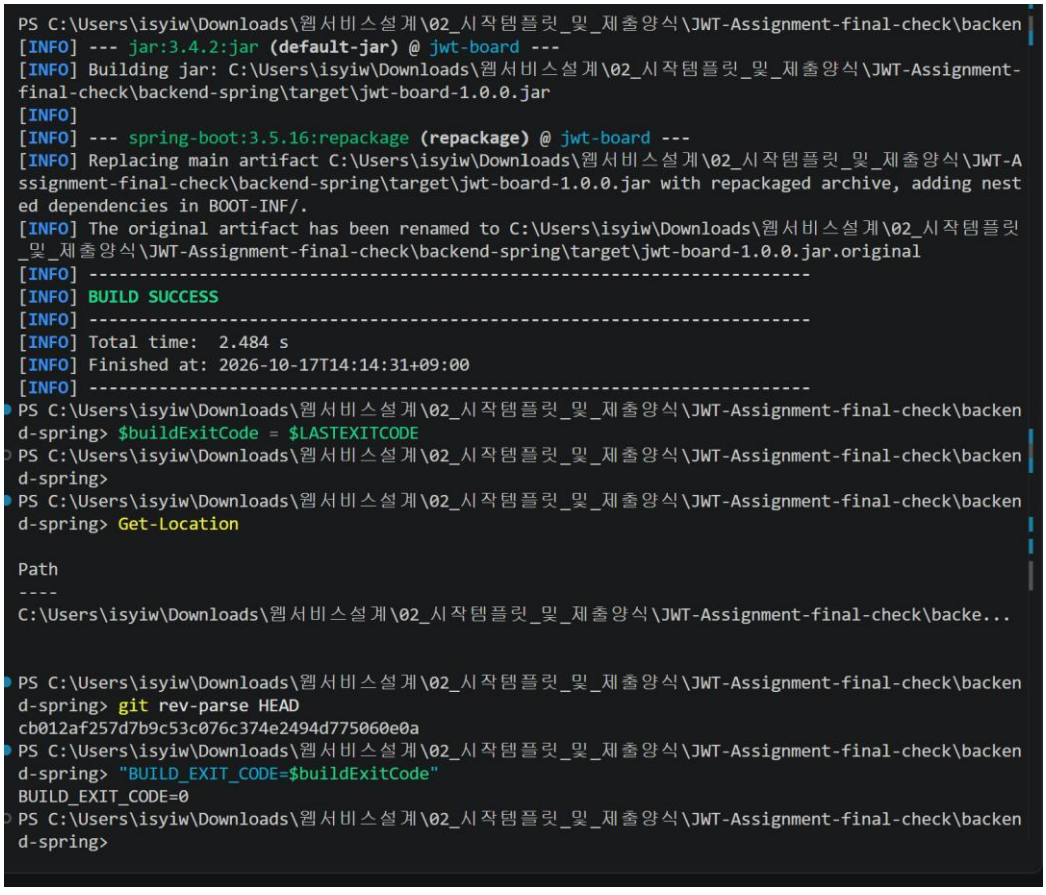


그림 11. 새 clone 폴더의 백엔드 빌드 — BUILD SUCCESS, 종료 코드 0

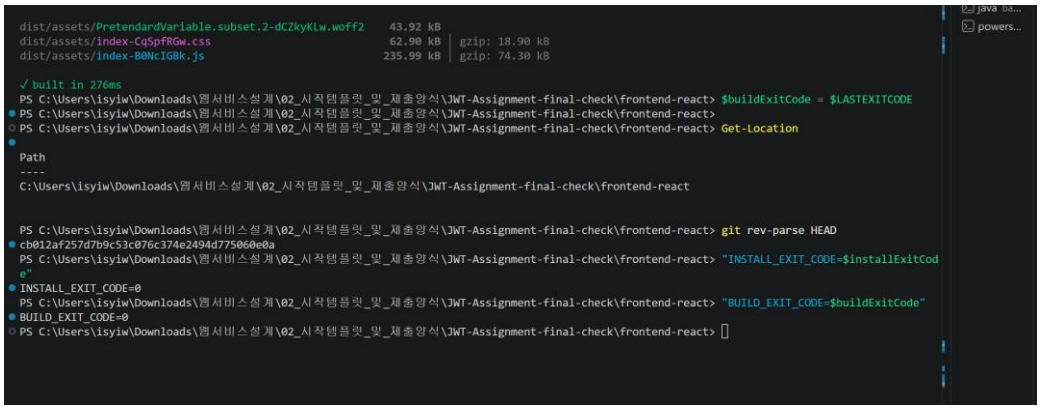


그림 12. 새 clone 폴더의 프론트엔드 설치·빌드 — npm.cmd ci 와 npm.cmd run build 의 종료 코드 모두 0



그림 13. 서버 재시작 — TOKEN_TTL=90 으로 복원한 뒤 8000 번 포트에서 기동 확인

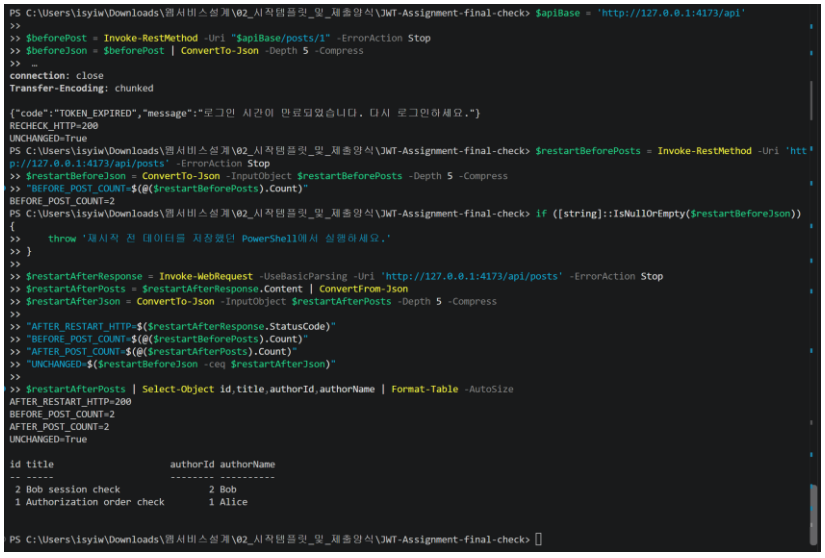


그림 14. 재시작 전후 데이터 비교 — 목록 조회 200, 게시글 수 전후 모두 2 개, 전체 응답 필드 비교 UNCHANGED=True

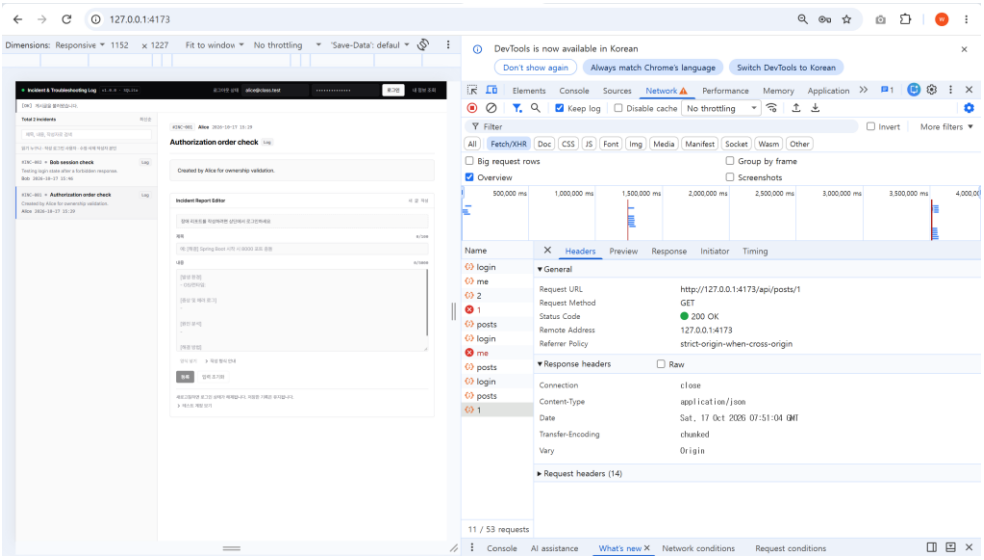


그림 15. 재시작 후 4173 번 화면 — Alice 의 1 번 글 조회 200, 기존 제목·본문·작성자 유지

웹서비스설계 05

AI 활용과 참고 자료

AI 사용 여부: 사용

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
Codex, 모델 미확인	JWT 발급·검증과 작성자 검사 구현, 프론트엔드 인증 연결, 오류 표시·반응형 수정, 최종 검수·추가 테스트 요청 명령 작성, README·보고서 초안과 교정	AuthService.java, BoardController.java 의 fields(), frontend-react/src/api.js, main.jsx, style.css, README.md, 보고서	코드와 README 는 Codex 가 작성·수정한 내용을 그대로 반영했다. 보고서는 초안을 참고해 직접 작성하고 교정했으므로 수정 후 반영에 해당한다. 테스트 실행과 캡처는 직접 수행했다.
Claude Code, 모델 미확인	장애·트러블슈팅 게시판 화면 초안 작성	frontend-react/src/main.jsx, src/style.css, index.html, package.json, package-lock.json	수정 후 반영. Claude Code 의 게시판 UI 초안에 Codex 가 레이아웃·오류 표시·반응형 스타일 수정을 적용했다. 화면 확인과 캡처는 직접 수행했다.

대표 사례

대표 사례 1. JWT 발급·검증과 작성자 권한 검사

도구 / 날짜: Codex / 초기 코드 작성 날짜는 미기록이다. 최종 검수와 추가 검증은 2026 년 10 월 17 일에 수행했다.

요청한 내용: 제공 템플릿의 미완성 인증을 수업 공지와 API 명세에 맞게 구현하도록 요청했다. 최종 검수에서는 비밀번호 원문 기준 입력 검사와 추가 테스트 방법을 정리하도록 요청했다. AI 가 적용한 코드 수정과 직접 수행한 테스트를 구분하도록 요청했다.

AI 가 제안한 내용: 64 자리 16 진수 비밀 키를 바이트로 변환해 HS256 서명에 사용하도록 제안했다. Bearer 헤더, 서명, 알고리즘, 만료, 필수 클레임과 계정 존재를 검사하도록 제안했다. 글 수정·삭제에서는 토큰의 계정 ID 와 저장된 작성자 ID 를 비교해 불일치 시 403 FORBIDDEN 을 반환하도록 제안했다.

실제 반영 위치: backend-spring/src/main/java/edu/classroom/board/AuthService.java 의 생성자, issueToken(), currentUserId(), verifiedUserId(), assertOwner()에 반영했다. BoardController.java 의 fields()에는 비밀번호 원문 기준의 빈 값·길이 검사 수정을 반영했다.

내가 확인하고 수정한 내용: AuthService 생성자의 키 형식·발급자·대상·TTL 검사와 토큰 검증 조건을 코드에서 확인했다. 정상 난수 키를 환경변수로 설정한 서버 기동은 M15-clone-backend-start.png 에 기록했다. 비밀 키 누락·형식 오류·예시 키에 대한 기동 실패 시험은 제출 증거로 확인되지 않아 실행 결과 미확인으로 구분했다. 숫자형 exp 검사와 비밀번호 원문 기준 입력 검사는 Codex 가 작성한 수정 코드를 반영했다. Users.java 의 PBKDF2 검증과 PostStore.java 의 CRUD 는 제공 코드를 유지했다.

검증 방법과 결과: M18-password-inputs.png 에서 빈 문자열 비밀번호는 400 BAD_INPUT, 공백 1 자와 정상 비밀번호 앞뒤에 공백을 추가한 입력은 각각 401 INVALID_CREDENTIALS, 정상 비밀번호는 200 임을 확인했다. M24-check-api-final.log 에서 일반 검사 40 개, M25-check-api-expiry.log 에서 만료 포함 검사 48 개가 통과했다. 서명 변조는 401 INVALID_TOKEN, 만료는 401 TOKEN_EXPIRED 이다. M22-1-bob-update-403.png 와 M22-2-bob-delete-403.png 에서는 Bob 의 타인 글 수정·삭제가 각각 403 FORBIDDEN 이며, 재조회는 200 이고 UNCHANGED=True 임을 확인했다.

최종 검수에서 비밀번호의 빈 값 검사와 길이 검사가 서로 다른 값을 사용하는 점을 확인했다. Codex 에 수정을 요청했고 Codex 가 BoardController.java 의 fields()에서 t.isEmpty()를 checked.isEmpty()로 변경했다. 비밀번호는 공백을 제거하지 않은 원문으로 빈 값과 길이를 검사한다. 이메일·제목·본문은 공백 제거 후 검사를 유지한다. 비밀번호 원문이 빈 문자열이거나 128 자를 초과하면 400 BAD_INPUT 이다.

대표 사례 2. 프론트엔드 토큰 관리와 오류 처리

도구 / 날짜: Codex / 초기 코드 작성 날짜는 미기록이다. 기존 수동 확인은 2026 년 10 월 15~16 일에, 401·403 화면의 추가 확인은 2026 년 10 월 17 일에 수행했다.

요청한 내용: 로그인 응답의 토큰을 메모리에 저장하고 보호 API 요청의 Authorization 헤더에 첨부하도록 요청했다. 실제 HTTP 상태와 JSON 오류 코드에 따라 알림을 표시하고, 401 에서는 로그인 상태를 해제하며 403 에서는 유지하도록 요청했다.

AI 가 제안한 내용: 모듈 변수 session 에 토큰과 계정 정보를 저장하고 authorizationHeaders()에서 Bearer 헤더를 생성하도록 제안했다. request()에서 401 응답일 때 clearSession()을 호출하고, 화면에서는 오류 객체의 status 와 code 를 사용하도록 제안했다.

실제 반영 위치: frontend-react/src/api.js 의 login(), authorizationHeaders(), request(), clearSession(), canEdit()에 반영했다. frontend-react/src/main.jsx 의 run(), save(), 로그인·로그아웃 이벤트에도 반영했다.

내가 확인하고 수정한 내용: Codex 가 작성한 토큰 저장·전송 코드와 오류 표시 코드를 반영했다. 알림에는 e.status 와 e.code 를 사용한다. 토큰은 session 에 저장하며 localStorage 와 sessionStorage 를 사용하지 않는다. 제공 코드의 204 빈 본문 처리를 유지했다. M03-input-400.png 에서는 400 BAD_INPUT 이후 작성 중인 입력이 남는 것을 확인했다.

검증 방법과 결과: M23-1-forbidden-session.png 에서는 한 번의 DELETE 요청 대상을 Bob 의 2 번 글에서 Alice 의 1 번 글로 바꾸어 실제 403 FORBIDDEN 을 발생시켰다. 권한 안내 배너가 표시되고 Bob 의 로그인 상태가 유지되었다. M27-expired-session-cleared.png 에서는 내 정보 요청의 401 TOKEN_EXPIRED 응답 후 만료 안내와 로그인 해제를 확인했다. M13-1-logout-headers.png 와 M13-1-logout-response.png 에서는 로그아웃 후 Authorization 헤더가 빠지고 401 TOKEN_REQUIRED 가 반환되는 것을 확인했다. M04-3-bob-delete.png 에서는 본인 글 삭제의 204 응답과 목록 갱신을 확인했다.

대표 사례 3. 게시판 화면 구성과 반응형 수정

도구 / 날짜: Claude Code·Codex / 코드 작성 날짜 미기록. 직접 화면을 확인한 날짜는 2026 년 10 월 15 일이다.

요청한 내용: 장애 기록을 목록과 상세 화면으로 나누고 발생 환경·증상·원인·해결 방법을 구분해 표시하도록 요청했다. 화면 개편 후에는 수정할 부분을 확인하도록 요청했다.

AI 가 제안한 내용: Claude Code 는 목록·상세 분할, 본문 구역 표시, 코드 블록, 검색과 해결 방법 복사 기능을 포함한 화면 초안을 작성했다. 이후 Codex 는 상단 바의 줄바꿈 기준을 1023px 이하로 조정하도록 제안했다.

실제 반영 위치: frontend-react/src/main.jsx, src/style.css, index.html, package.json, package-lock.json 에 화면 초안을 반영했다. 상단 바의 줄바꿈 수정은 frontend-react/src/style.css 에 반영했다.

내가 확인하고 수정한 내용: Claude Code 의 화면 초안을 반영한 뒤 Codex 의 후속 수정 코드를 적용했다. 상단 바 줄바꿈은 1023px 이하, 목록·상세의 세로 배치는 899px 이하로 구분했다. 직접 너비 900px 과 390px 에서 화면 배치를 확인했다.

검증 방법과 결과: 2026 년 10 월 15 일에 너비 900px 과 390px 에서 화면 배치를 확인했다. M09-responsive-900.png 에는 900px 화면의 상단 바 줄바꿈을, M10-responsive-mobile.png 에는 390px 화면의 목록·상세 세로 배치를 기록했다. 2026 년 10 월 17 일에는 새 clone 폴더에서 프론트엔드 설치와 빌드를 다시 수행했다. M16-clone-frontend-build.png 에서 npm.cmd ci 와 npm.cmd run build 의 종료 코드가 모두 0 임을 확인했다.

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
docs/01_과제공지.md	구현 범위, 보고서 01·04 절	필수 기능, JWT 조건, 제출물과 기본 포트의 기준으로 사용했다.
docs/02_API 명세.md	BoardController.java, AuthService.java, frontend-react/src/api.js	요청 경로·필드, HTTP 상태, JSON 오류 코드와 응답 헤더를 대조했다.
docs/03_구현체크리스트.md	인증·권한 구현 확인	백엔드와 프론트엔드의 구현 항목을 확인했다.
docs/04_보고서양식.docx, docs/04_보고서양식.md	보고서 01~06 절	절 구성, 코드 발췌, 테스트 기록과 AI 활용 출처 형식을 따랐다.

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
docs/05_테스트시나리오.md	보고서 04 절	T01~T17 의 절차·기대 응답과 실제 결과를 대조했다.
docs/07_로컬실행안내.md	README.md, 로컬 실행 기록	설치·환경변수·빌드·preview·JAR 실행과 SQLite 경로를 정리했다.
tools/check_api.py	일반 검사·만료 검사	제공 파일을 유지한 채 직접 실행했다. 일반 검사 결과는 M24-check-api-final.log 와 M24-check-api-final.png 에, 만료 포함 검사 결과는 M25-check-api-expiry.log 와 M25-check-api-expiry.png 에 기록했다.
backend-spring/src/main/, frontend-react/src/	보고서 02·03·06 절	실제 함수·조건문·변수와 코드 발췌를 대조했다.
docs/evidence/manual/	보고서 04 절	로그인, 요청 헤더, HTTP 응답, 빌드와 서버 재시작 결과를 기록했다.
https://github.com/orioncactus/pretendard/blob/v1.3.9/LICENSE`	frontend-react/src/main.jsx, frontend-react/src/style.css	main.jsx 에서 글꼴 CSS 를 가져오고 style.css 에서 화면 글꼴로 지정했다. 라이선스는 SIL Open Font License 1.1(OFL-1.1)이다.

이해도 확인과 회고

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

요청 본문의 authorId 를 작성자 ID 로 사용하면 Bob 이 authorId: 1 을 보내 Alice 이름으로 글을 등록할 수 있다. BoardController.java 의 create()는 auth.currentUserId(req)로 확인한 계정 ID 를 store.create()에 전달한다. 작성자는 이 값으로 결정한다. 요청 본문은 fields()에서 title 과 content 만 허용한다. authorId 나 userId 를 추가하면 DB 저장 전에 거부하며, 응답은 400 BAD_INPUT 이다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

frontend-react/src/api.js 의 canEdit()는 로그인 계정의 ID 와 post.authorId 가 일치하는지 확인하고, main.jsx 는 그 결과에 따라 수정·삭제 버튼을 표시한다. M04-bob-no-actions.png 에서 Bob 이 Alice 글을 조회할 때 수정·삭제 버튼이 표시되지 않는 것을 확인했다. 버튼이 없어도 API 클라이언트로 PUT 이나 DELETE 요청을 직접 보낼 수 있으므로 서버에서도 권한을 검사해야 한다. BoardController 의 update()와 delete()는 인증과 글 조회 후 AuthService.assertOwner()를 호출한다. 이 함수는 검증된 계정 ID 와 DB 에서 조회한 작성자 ID 를 비교하며, 다르면 DB 를 변경하기 전에 403 FORBIDDEN 을 반환한다. M22-1-bob-update-403.png 와 M22-2-bob-delete-403.png 에서 실제 403 응답과 재조회 200, UNCHANGED=True 를 확인했다.

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

decode 는 토큰의 헤더와 본문을 해석해 내용을 읽는 작업이다. 내용을 읽는 것만으로 서명이나 클레임의 유효성을 확인할 수는 없다. 내 AuthService.java 는 parser.parseSignedClaims()로 서명을 검증하며 HS256 만 허용한다. iss 는 설정된 발급자와 일치해야 하고, aud 에는 설정된 대상이 포함되어야 한다. exp 의 존재·숫자형·만료 여부를 확인한다. sub 는 양의 정수를 나타내는 문자열이어야 하며, long 으로 변환한 ID 에 해당하는 계정이 실제로 존재해야 한다. 헤더 누락·형식 오류는 401 TOKEN_REQUIRED, 서명·알고리즘·필수 클레임·계정 검증 실패는 401 INVALID_TOKEN 이다.

만료 예외가 발생해도 verifiedUserId(e.getClaims())로 필수 클레임과 계정 존재를 다시 검사한다. 나머지 조건을 통과한 만료 토큰의 응답은 401 TOKEN_EXPIRED 이다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃은 frontend-react/src/api.js 의 clearSession()을 호출한다. main.jsx 의 로그아웃 이벤트는 화면의 계정·프로필·수정 상태도 초기화한다. 이 탭의 변수 상태는 session = { accessToken: "", user: null }이다.

토큰 만료 시각이 지났다고 브라우저 상태가 즉시 바뀌지는 않는다. 만료 토큰으로 보호 API 를 요청하면 서버가 401 TOKEN_EXPIRED 를 반환한다. api.js 의 request()가 clearSession()을 호출한 뒤 main.jsx 의 run()이 화면 상태를 갱신한다. 처리 후 변수 상태는 session = { accessToken: "", user: null }이다.

새로고침하면 api.js 모듈을 다시 불러오므로 로그인 정보가 초기화된다. SQLite 에 저장된 게시글은 삭제되지 않는다. M05-refresh.png 에서 로그아웃 상태와 기존 글 표시를 확인했으며, 글 조회 응답은 200 이다.

서버에는 로그인 세션이나 토큰 폐기 목록을 저장하지 않는다. 로그아웃도 서버에 토큰 폐기 요청을 보내지 않는다. 따라서 이미 외부로 복사한 JWT 는 서명 키·클레임·계정이 계속 유효하면 exp 전까지 인증에 사용할 수 있다. 다른 검증 조건을 통과하더라도 만료 시각이 지나면 응답은 401 TOKEN_EXPIRED 이다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

화면 개편 후 너비 900px 에서 상단 바의 버튼이 화면 밖으로 나오는 문제를 확인했다. 로그인 입력란과 버튼이 가로로 배치된 상태에서 줄바꿈이 적용되지 않아 좁은 화면에 대응하지 못하는 문제였다. Codex 에 상단 바의 줄바꿈 조건과 목록·상세의 배치 전환 조건을 분리하도록 요청하고, 작성된 수정 코드를 반영했다. frontend-react/src/style.css 는 1023px 이하에서 .topbar, .topbar-right, .auth-bar 에 flex-wrap: wrap 을 적용한다. 목록·상세는 899px 이하에서 한 열로 배치한다. 수정 후 직접 900px 과 390px 화면을 확인했으며, M09-responsive-900.png 에는 상단 바 줄바꿈을, M10-responsive-mobile.png 에는 목록·상세의 세로 배치를 기록했다. 2026 년 10 월 17 일에는 새 clone 폴더에서 프론트엔드를 다시 설치·빌드했고, M16-clone-frontend-build.png 에서 두 명령의 종료 코드가 모두 0 임을 확인했다.

현재 남은 문제와 다음에 개선할 점: 기능 테스트에서 확인된 오류는 없음. 추가 시험의 미실행·미확인 사항은 04·05 절에 기록했다.

제출 확인: [v] 실제 코드와 설명 일치 [v] 테스트 직접 수행 [v] AI 활용 및 출처 기록 [v] 비밀 키와 토큰 제거

GitHub 확인: [v] 루트 README 작성 [v] 보고서 PDF 와 증거 업로드 [v] README 문서 링크 확인 [v] 교수자 접근 가능 [v] 저장소 URL 과 커밋 SHA 제출